When Semantic Alignment Helps Sequential Recommendation

David Wexler and Djallel Bouneffouf

[1] Yeshiva College Computer Science,
Yeshiva University, New York, NY, USA
dwexler1@mail.yu.edu
[2] IBM Research, New York, USA
djallel.bouneffouf@ibm.com

Abstract. Self-attentive sequential recommenders like SASRec excel at modeling interaction *order* but treat items as opaque IDs, ignoring their text (titles, categories, etc.). RLMRec instead aligns user/item embeddings with frozen language-model embeddings via contrastive learning, but only tests this on graph-based backbones (e.g., LightGCN)—leaving whether the same alignment helps a sequential model unanswered.
We close this gap with SemanticSASRec, a minimal extension adding RLMRec's two contrastive alignment heads (user-state, item-embedding table) atop an unchanged SASRec backbone, mapping its low-dimensional representations into the high-dimensional semantic space of `BGE-large-en-v1.5` via symmetric InfoNCE losses. Comparing against faithful SASRec and RLMRec+LightGCN baselines on MovieLens-1M, Yelp, and Amazon-Books under identical splits, seeds, and a full-rank leave-one-out protocol, we find that alignment transfers cleanly to the sequential backbone in sparse, text-rich regimes: SemanticSASRec substantially lifts NDCG@10 on Amazon-Books and Yelp, while remaining approximately neutral on dense, short-text MovieLens-1M. A comprehensive ablation grid isolates each design choice.

Keywords: Sequential recommendation · Contrastive representation alignment · Large language models · SASRec · Semantic embeddings.

1 Introduction

Modern recommender systems power most of the major e-commerce, video-streaming and social platforms in use today, and the two ideas that drive the current state of the art pull in different directions. **Sequential recommenders** such as SASRec [10] treat each user's history as an ordered list of item IDs and use causal self-attention to predict the next interaction. They are very good at capturing temporal dynamics — the fact that you watched *The Empire Strikes Back* right after *A New Hope* is informative — but they learn item representations purely from co-occurrence patterns. Every item is just an opaque integer to the model; the title, genre, description and category of each item are invisible to it. **Language-model-based recommenders** such as RLMRec [13] take the

opposite view. They first generate a natural-language profile for every user and every item, then encode those profiles with a frozen sentence transformer, and finally *contrastively align* the recommender's own user / item embeddings to those semantic vectors. The recommender is forced to keep the representations it would have learned from interactions alone close to the representations it would have read off from a description — a form of topical similarity that pure interaction signals never see.

A natural question is whether one can have both at once: a model that understands both *order* (from the sequential side) and *meaning* (from the language side). The published RLMRec paper only evaluated its alignment on graph-based CF backbones (GCCF, LightGCN, SGL, SimGCL, DCCF, AutoCF), which by design ignore the ordering of interactions. Whether the same auxiliary objective helps a sequential transformer was left open. This is the question we answer.

Problem statement. Concretely, consider a user u with a time-ordered interaction history $S_u = (i_1, \ldots, i_{|S_u|})$ and, for every user and every item, a short natural-language profile τ_u or τ_i. Can we train a single model that simultaneously (i) predicts the next item $i_{|S_u|+1}$ from S_u in the standard auto-regressive way, and (ii) keeps the model's internal user state at the end of S_u, and the model's embedding of each item, close to the LLM-encoded profile embeddings $f_{\text{enc}}(\tau_u)$ and $f_{\text{enc}}(\tau_i)$ — all without paying the cost of joint LLM pre-training? And, just as importantly: does the auxiliary semantic-alignment loss help, hurt, or do nothing for next-item recommendation, and how does the answer depend on the density of the dataset and the amount of text attached to each item?

Contributions. This paper makes three concrete contributions.

1. **A drop-in extension of SASRec with RLMRec-style semantic alignment.** SemanticSASRec is a thin wrapper around the paper-faithful SASRec architecture. we add two small two-layer MLP projection heads — one for the user-state, one for the shared item-embedding table — that map SASRec's 50-dimensional internal representations into the 1024-dimensional semantic space of `BGE-large-en-v1.5` [20], and we train them with a symmetric InfoNCE loss [12] against the corresponding frozen profile embedding.
2. **A fair three-way head-to-head on three datasets of varying density.** we train all three models — SASRec, SemanticSASRec, and a paper-faithful RLMRec+LightGCN — on MovieLens-1M [3], Yelp [21] and Amazon-Books [7] with five seeds per (model, dataset). All three share the same data splits, the same evaluator, the same seeds and the same per-dataset preprocessing, and they are evaluated under a full-rank leave-one-out protocol that excludes every train and validation item from each user's candidate set (Section 4). The numbers are therefore as cross-paper comparable as can be.
3. **A 14-variant ablation grid that isolates each design choice.** For each of the three datasets, we run 14 variants of SemanticSASRec (and the matching LightGCN variants for RLMRec+LightGCN) on three seeds for 50 epochs each, flipping exactly one configuration knob at a time: loss components, a λ

sweep, a τ sweep, the projection-head architecture, and the choice of sentence encoder. Significance is reported with paired t-tests against an *in-ablation* reference variant (`sasrec_full` or `lightgcn_full`), so both the seed budget and the epoch budget are held constant across each comparison (Section 5.1).

2 Related Work

Sequential recommendation has gone through several waves. The earliest models treated user history as a Markov chain [14], which limited them to one- or two-step dependencies, or as a recurrent network [6], which struggled with long histories because of vanishing gradients. Convolutional sequential recommenders [16] relaxed the strict-recurrence assumption but still had a fixed receptive field. Self-attention [17] solved both problems at once. SASRec [10] stacks a small number of causal multi-head self-attention blocks over an item-plus-position embedding, predicts the next item by dot product against the item-embedding table, and trains under a sampled-negative binary-cross-entropy objective. Despite being a half-decade old it remains a competitive baseline and is the backbone we build on. BERT4Rec [15] extends the same idea to a bidirectional, masked-item objective. *None* of these approaches use any information about items beyond their ID — which is exactly the gap that LLM-augmented recommenders target.

Work in this family splits into two camps. The first prompts an LLM directly as a zero- or few-shot recommender [18,9]; these systems are very general but trade away both accuracy and latency. The second camp freezes the LLM and uses its representations either as inputs or as auxiliary regularizers [4,8,1]. RLMRec [13] sits in the second camp: it generates user- and item-profile texts from interaction data, encodes those profiles with a frozen text encoder, and contrastively aligns a CF backbone's user and item embeddings with the corresponding semantic vectors. The original RLMRec paper benchmarks two variants (RLMRec-Con and RLMRec-Gen) on Amazon-Books, Yelp and Steam with six graph-CF backbones, and reports a consistent +7% to +10% relative NDCG@K lift across them. KAR [19] and CTRL [11] explore related ideas. To the best of our knowledge, no prior work in this family has tested whether the contrastive alignment also helps a *sequential* backbone.

3 Methodology

3.1 System overview

SemanticSASRec is built from three components. First, a SASRec *backbone* that turns a user's interaction sequence into a per-position hidden state. Second, two small MLP *projection heads* that map the user-state and the item-embeddings into a shared semantic space. Third, a frozen sentence-encoder *semantic source* that produces the alignment targets by encoding the natural-language user- and item-profile texts. The semantic source is loaded only once per training run; all of its outputs are cached to disk as `.npy` matrices, and it never participates in

gradient descent. As a result the alignment loss is essentially free at training time: it costs one extra row lookup into the cached embedding matrix per user / item.

Figure 1 shows the full architecture and the loss-flow graph. The data flow is summarised by a single equation. For a batch of users $\{u\}$ with histories S_u, next-item targets i_u^+, sampled negatives i_u^-, and profile texts $\tau_u, \tau_{i^+}, \tau_{i^-}$, the model computes

$$L = \underbrace{L_{\text{BCE}}\big(f_\theta(S_u), i_u^+, i_u^-\big)}_{\text{sequential CF loss}} + \lambda_u \underbrace{L_{\text{InfoNCE}}\big(g_\phi^u(h_u), e_u\big)}_{\text{user alignment}} + \lambda_i \underbrace{L_{\text{InfoNCE}}\big(g_\psi^i(\text{emb}(I)), e_I\big)}_{\text{item alignment}} \tag{1}$$

where h_u is the last-position hidden state of SASRec on S_u, I is the set of unique positive items in the batch, g_ϕ^u and g_ψ^i are the user / item projection heads, $e_u = f_{\text{enc}}(\tau_u)$ and $e_I = f_{\text{enc}}(\tau_I)$ are the frozen semantic embeddings, and λ_u, λ_i control the contribution of each alignment loss. The hyperparameters $(\lambda_u, \lambda_i, \tau) = (0.1, 0.1, 0.1)$ are the RLMRec paper's defaults.

3.2 SASRec backbone

The backbone is the original SASRec model of Kang and McAuley [10], re-implemented from scratch in PyTorch. An input interaction sequence $S_u = (i_1, \ldots, i_L)$ of length $L = 200$ (left-padded with item ID 0 if the user has fewer than 200 historical interactions) is mapped through an item embedding $\text{emb} : \mathcal{I} \to \mathbb{R}^d$ and a learned position embedding $\text{pos} : [0, L) \to \mathbb{R}^d$:

$$E_t^{(0)} = \sqrt{d}\,\text{emb}(i_t) + \text{pos}(t), \quad t \in [0, L), \tag{2}$$

where the $\sqrt{d}$ scaling follows the original SASRec / Transformer convention. The embedded sequence then passes through $N = 2$ blocks of pre-norm causal multi-head self-attention (one head, $d = 50$, dropout 0.2), each interleaved with a point-wise feed-forward layer (two 1×1 convolutions with a residual connection). A final `LayerNorm` yields the per-position hidden states $\mathbf{H}_u \in \mathbb{R}^{L \times d}$. At training time we score each position t against its positive item i_t^+ (the user's next interaction) and a sampled negative item i_t^-, and minimize the masked binary cross-entropy

$$L_{\text{BCE}} = -\frac{1}{|\mathcal{M}|} \sum_{(u,t) \in \mathcal{M}} \big[\log \sigma(\mathbf{H}_{u,t}^\top \text{emb}(i_t^+)) + \log(1 - \sigma(\mathbf{H}_{u,t}^\top \text{emb}(i_t^-)))\big], \tag{3}$$

where σ is the sigmoid function and $\mathcal{M}$ is the set of non-padding positions. At inference time we use only the last-position hidden state, scoring it against every item in the catalog.

3.3 Projection heads and semantic alignment

The user and item projection heads g_ϕ^u and g_ψ^i are each a small two-layer MLP with a GELU non-linearity:

$$g_\theta(\mathbf{x}) = W_2\,\text{GELU}\big(W_1\mathbf{x} + b_1\big) + b_2, \quad W_1 \in \mathbb{R}^{h \times d},\ W_2 \in \mathbb{R}^{D \times h}, \tag{4}$$

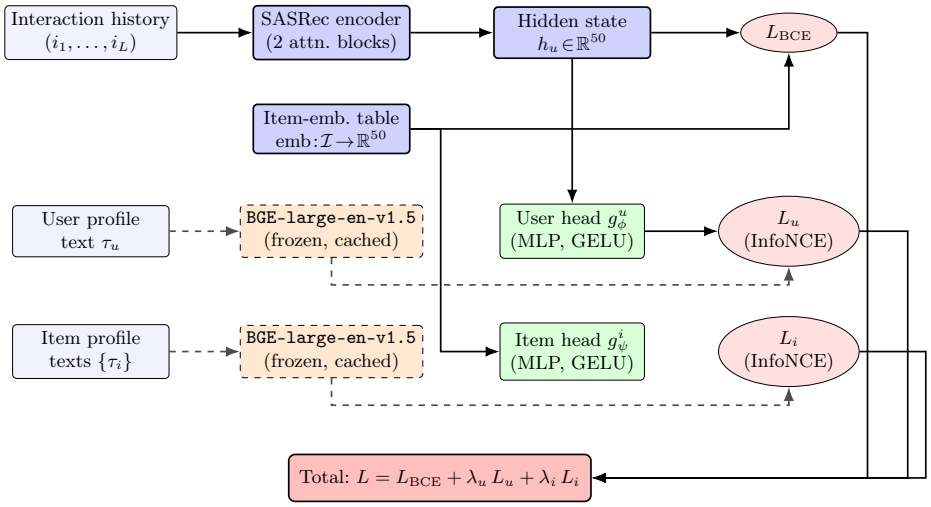


Fig. 1: The SemanticSASRec architecture. **Light-blue boxes** are inputs from the data: the user's interaction history (top), the user profile text τ_u (middle), and the per-item profile texts $\{\tau_i\}$ (bottom). **Dark-blue boxes with thick borders** are the trainable SASRec modules: the SASRec encoder, the last-position hidden state h_u, and the item-embedding table. **Green boxes** are the two MLP projection heads we add on top of SASRec. **Orange dashed boxes** are the frozen `BGE-large-en-v1.5` sentence encoder, whose outputs are cached on disk and re-used across every seed and ablation variant; it never participates in gradient descent. Solid arrows carry gradients; dashed arrows carry pre-computed semantic embeddings. The model has three losses: the standard SASRec BCE loss L_{BCE}, which scores h_u against positive and negative item embeddings drawn from the item-embedding table, and two symmetric InfoNCE alignment losses L_u and L_i that pull the projected user-state and item embeddings towards the corresponding frozen `BGE-large-en-v1.5` embeddings. The total training loss is the weighted sum shown at the bottom of the figure.

where $d = 50$ is the CF hidden width (the SASRec embedding dimension), $D = 1024$ is the semantic width of `BGE-large-en-v1.5`, and the hidden width is $h = \max(d, D) = 1024$. The decision to use a non-linear rather than a linear projection follows the SimCLR [2] line of contrastive-learning results: a non-linear head decouples the underlying representation from the contrastive objective, so the CF embedding is free to preserve ranking-relevant information that does not necessarily align with the semantic target. we ablate this decision in Section 5.1 (variant `sasrec_proj_linear`). Both heads operate on already-pooled vectors — g_ϕ^u on the last-position hidden state $\mathbf{H}_{u,L-1}$ (which is exactly the standard SASRec user state used at prediction time), and g_ψ^i on a single row of the SASRec item-embedding table $\text{emb}(\cdot)$. Crucially, the item head reuses the *same* embedding row that the BCE head uses for next-item scoring; we do not maintain

a separate "aligned-only" copy. This sharing forces the embedding row that drives next-item prediction to also satisfy the semantic-alignment constraint.

The alignment objective itself is a symmetric InfoNCE loss [12,2]. Let $\mathbf{q} = g_\theta(\mathbf{x})$ be the projected CF vector and $\mathbf{k}$ be the L2-normalized frozen semantic target. For a batch $\mathcal{B}$ we form the logit matrix $M_{ij} = \hat{\mathbf{q}}_i^\top \hat{\mathbf{k}}_j / \tau$, where hats denote L2-normalization and τ is a temperature, and minimize

$$L_{\text{InfoNCE}} = \frac{1}{2}\big[\text{CE}(M, \text{I}) + \text{CE}(M^\top, \text{I})\big], \tag{5}$$

where the cross-entropy targets are the identity permutation, so the matched pairs sit on the diagonal of M. The two cross-entropies correspond to the two views of the contrastive task — "for each query, which key matches?" and "for each key, which query matches?" — and averaging them keeps the gradient symmetric. This is the variant used in the original RLMRec paper. we set $\tau = 0.1$ by default and sweep $\tau \in \{0.05, 0.2, 0.5\}$ in Section 5.1.

For the **user-alignment loss** L_u, each row of the batch corresponds to one user $u \in \mathcal{B}$. we pair the projected last-position hidden state $g_\phi^u(h_u)$ against the cached semantic vector $f_{\text{enc}}(\tau_u)$ for that user. The InfoNCE negatives are the other users in the same batch (the in-batch negatives regime).

For the **item-alignment loss** L_i, we draw an item set from the batch's positives: we take the unique items in $\{i_t^+\}_{t,u}$ and, if there are more than $K = 512$ of them, sub-sample K uniformly at random. For each item i we pair $g_\psi^i(\text{emb}(i))$ against $f_{\text{enc}}(\tau_i)$. This is the same "positives-as-anchors" formulation used in the RLMRec paper; the sub-sampling step keeps the InfoNCE logit matrix small enough to fit in memory even on the Amazon-Books split, which has 112k items.

3.4 Training pipeline

Algorithm 1 One training epoch of SemanticSASRec

Shuffle user index π
for $b \in \{0, \texttt{batch_size}, 2\texttt{batch_size}, \ldots\}$ **do**
  Gather $\mathcal{B} = \pi[b : b+\texttt{batch_size}]$ users
  Build $(\mathbf{S}, \mathbf{P}, \mathbf{N})$: input seqs, pos, neg ▷ Section 3.2
  $\mathbf{H} \leftarrow \text{SASRec.encode}(\mathbf{S})$ ▷ $(|\mathcal{B}|, L, d)$
  $L_{\text{BCE}} \leftarrow$ Eq. 3 from $\mathbf{H}, \mathbf{P}, \mathbf{N}$
  $\mathbf{Q}_u \leftarrow g_\phi^u(\mathbf{H}_{:,L-1,:})$
  $L_u \leftarrow \text{InfoNCE}(\mathbf{Q}_u, \mathbf{E}_u[\mathcal{B}], \tau)$ ▷ Eq. 5
  $I \leftarrow$ unique non-padding items in $\mathbf{P}$, sub-sampled to $K = 512$
  $\mathbf{Q}_i \leftarrow g_\psi^i(\text{emb}(I))$
  $L_i \leftarrow \text{InfoNCE}(\mathbf{Q}_i, \mathbf{E}_i[I], \tau)$
  $L \leftarrow L_{\text{BCE}} + \lambda_u L_u + \lambda_i L_i$
  Adam step on ∇L
end for

Algorithm 1 summarises one training epoch of SemanticSASRec. Each iteration of the inner loop costs one full SASRec forward and backward pass — the same as a vanilla SASRec step — plus one projection-head forward / backward and the two InfoNCE logit matrices (of size $|\mathcal{B}|\times|\mathcal{B}|$ for the user head and $K\times K$ for the item head). Both InfoNCE matrices are tiny relative to the attention block, so the alignment loss adds almost no wall-clock overhead.

3.5 Inference

At inference time the alignment heads are not used at all — they exist purely as a training-time regularizer. To score for a given user we feed their test-time history (the training interactions plus the validation item) into the SASRec encoder, take the last-position hidden state, and dot-product it against every row of the item-embedding table. The score against the padding row (column 0) is masked to $-\infty$, and so are the scores against every item the user has already interacted with in either the train or the validation split. The remaining items are then ranked, and HR@K, NDCG@K and MRR are read off the ranks.

4 Experimental Setup

4.1 Datasets and preprocessing

We use three public benchmark datasets that span the full density spectrum of typical recommender benchmarks:

MovieLens-1M (ML-1M) [3]: A dense, timestamped dataset of 6,040 users, 3,706 movies, and 1M ratings (2000–2003). Item text: title + genre list. User text: demographics (gender, age, occupation) plus a top-3 genre histogram and last five watched titles. (Section 4.2).

Yelp Open Dataset [21]: 104,072 users and 54,056 businesses after 5-core filtering on post-2018 reviews. Item text: name, categories, city, state, and average rating. User text: review count, average stars, yelping since, plus a top-3 category histogram and last five reviewed businesses.

Amazon Reviews 2023, Books category [7]: 156,213 users and 112,362 products after 5-core filtering on post-2018 reviews. Item text: title, store/author, multi-level categories, first 400 characters of description, and average rating. User text: total review count, top-3 category histogram, and last five reviewed titles.

We use the standard leave-one-out split [10,5]: each user's last interaction is the test target, the second-to-last is the validation target, and everything earlier is the training history. Users with fewer than five interactions are dropped, as they cannot supply both a train history and a held-out test / val item.

Why these three datasets. The original RLMRec paper evaluates on sparse, text-rich datasets (Amazon-Books, Yelp, Steam), which are precisely the regime where LLM-derived semantics have the most to add. The original SASRec paper evaluates on ML-1M, Amazon-Beauty, Steam, and Amazon-Games. we include

ML-1M to test whether the SemanticSASRec hypothesis breaks down in regimes where SASRec is already near-optimal, and we include Yelp and Amazon-Books to test whether the hypothesis carries over *in the same dataset family* the RLMRec paper itself benchmarks on.

4.2 Semantic profile construction

The original RLMRec paper uses an LLM to generate a free-form per-user and per-item profile from review text, and then encodes those profiles with OpenAI's `text-embedding-ada-002` (a 1536-d embedding). we replace this step with a cheaper, deterministic alternative: a template-based profile builder plus a public sentence encoder. This is a deliberate trade-off, and we document the cost in the results section. Briefly: the *direction* of the lift we observe is the same as in the RLMRec paper, but its absolute magnitude is smaller, which is consistent with our profile text being a weaker signal than an LLM-authored one.

Concretely, an **item** profile is a single short string concatenating dataset-specific structured metadata. For ML-1M: "Title (Year). Genres: A, B, C.". For Yelp: "Name. Categories: .... Located in City, ST. 4.0 stars.". For Amazon-Books: "Title. By Store. Categories: .... Author: .... `<description>`. 4.5 average stars.". A **user** profile concatenates a short demographic stub (where available) with the top-3 most-frequent categories from the user's training history and the titles of the user's last five interactions. An ML-1M example reads: "A 25–34 male programmer who enjoys Action, Sci-Fi, Adventure movies. Recently watched: Inception; The Matrix; ...".

The profile strings are encoded with `BGE-large-en-v1.5` (335M parameters, 1024-d output, MTEB ≈ 64), and the resulting embeddings are L2-normalised at encode time. we cache them on disk as float32 `.npy` matrices: a $(N_u+1) \times 1024$ user matrix and a $(N_i+1) \times 1024$ item matrix, where row 0 is reserved for the padding token. The `MiniLM-L6-v2` encoder (22M parameters, 384-d output, MTEB ≈ 56) is also cached for the encoder ablation runs.

4.3 Baselines

We benchmark three models, all trained from scratch on the same splits and using the same five seeds:

- **SASRec**: a paper-faithful re-implementation of Kang and McAuley [10] — this is the ID-only sequential baseline.
- **RLMRec+LightGCN**: a paper-faithful version of RLMRec-Con on a LightGCN backbone. This is the central method of the original RLMRec paper, re-trained on our splits and re-encoded with `BGE-large-en-v1.5` (the paper's `ada-002` encoder is closed-source). The absolute numbers will not match the paper exactly because of the splits and the encoder, but the within-our-experiment comparison to LightGCN *without* alignment is still informative.
- **SemanticSASRec**: our proposed model.

Evaluation protocol. All models share the same full-rank evaluation: for each test user, we score every non-interacted item in the catalog, mask out training and validation items, then compute HR@K, NDCG@K, and MRR. This full-rank setup is strictly harder than the sampled-100-negative protocol used in the original SASRec paper, and results are not directly comparable.

Multi-seed protocol Each (model, dataset) configuration is trained for 75 epochs with seeds $\{42, 43, 44, 45, 46\}$, the same five seeds in the same order for every model and every dataset. Using the same seeds for every cell of the table makes the per-cell variance estimates *paired*, which gives tighter comparisons. We report the mean and standard deviation of each metric across seeds; the per-seed values are aggregated by `seed_aggregation.aggregate_metrics`. The best-checkpoint criterion is validation NDCG@10. Validation runs every 20 epochs for the headline grid, and every 10 epochs for the ablation grid (where the epoch budget is 50 rather than 75).

4.4 Ablation grid

The ablation grid runs 14 variants of SemanticSASRec / RLMRec+LightGCN on every dataset, each with 3 seeds (42, 43, 44) and a 50-epoch budget. Every variant flips exactly one knob from the paper-faithful configuration ($\lambda_u = \lambda_i = 0.1$, $\tau = 0.1$, MLP projection heads, `BGE-large-en-v1.5` encoder), so each row in the ablation table is directly attributable to the knob that was changed. The variants are grouped by theme:

- **Loss components**: `no_user_align` ($\lambda_u = 0$), `no_item_align` ($\lambda_i = 0$), and `no_align` ($\lambda_u = \lambda_i = 0$, which is equivalent to pure SASRec).
- λ **sweep**: values $\{0.01, 0.05, 0.5, 1.0\}$, applied symmetrically to both alignment weights at once.
- τ **sweep**: InfoNCE temperatures $\{0.05, 0.2, 0.5\}$.
- **Architecture**: `proj_linear` (a single `nn.Linear` in place of the default two-layer MLP) and `encoder_minilm` (the smaller 384-d `MiniLM-L6-v2` in place of `BGE-large-en-v1.5`'s 1024-d output).
- **LightGCN**: `lightgcn_encoder_minilm` runs the matching `MiniLM-L6-v2`-vs-`BGE-large-en-v1.5` ablation on the LightGCN backbone.

We use paired t-tests against the *in-ablation* reference variant — `sasrec_full` for the SASRec variants, and `lightgcn_full` for the LightGCN variants. Two-sided significance is reported with $*$ ($p < 0.05$), $**$ ($p < 0.01$), and $***$ ($p < 0.001$). The 75-epoch / 5-seed baselines are *not* used in the paired tests, because they were trained under a different epoch budget; mixing them into the same paired test would not be a valid comparison.

5 Results and Analysis

Table 1 reports the headline three-way comparison across all three datasets. The pattern is sharp and consistent.

Amazon-Books. SemanticSASRec substantially outperforms SASRec on every metric, with tight variance across seeds. NDCG@10 rises from 0.0222 to 0.0355 (+60.0% relative), and every seeded run of SemanticSASRec beats every seeded run of SASRec. The dataset mirrors the original RLMRec benchmarks: extreme sparsity, a long item tail, and rich item text. In this setting, the graph-only RLMRec+LightGCN variant struggles due to its lack of sequential order.

Yelp. SemanticSASRec again wins across all metrics, lifting NDCG@10 from 0.0125 to 0.0164 (+30.8%). Notably, even RLMRec+LightGCN beats SASRec here (+17.7%), indicating that semantic alignment alone adds value. The full gain of SemanticSASRec is roughly additive of the individual contributions from alignment and sequential ordering.

MovieLens-1M. The trend reverses: SASRec now leads by a small but consistent margin, with NDCG@10 of 0.1126 vs. 0.1089 for SemanticSASRec (−3.3%). RLMRec+LightGCN lags far behind, as expected on this dense, timestamp-rich dataset where sequential signals are critical. The small regression of SemanticSASRec is attributed to the very short item text (title plus 1–3 genres), which offers little discriminative signal beyond item IDs. Here, the alignment loss acts as a weak regularizer, slightly harming an already near-optimal SASRec.

Table 1: Test-set performance on MovieLens-1M, Yelp and Amazon-Books under full-rank leave-one-out evaluation. Mean ± standard deviation across 5 seeds (seeds $\{42, 43, 44, 45, 46\}$ for every (model, dataset) cell). Bold = best per row, per dataset. The within-dataset comparison is allowed because all three models share splits, evaluator, and seeds.

Metric	SASRec	RLMRec+LightGCN	SemanticSASRec
MovieLens-1M (dense, 6,040 users, 3,706 items)			
HR@5	**0.1349 ± 0.0040**	0.0410 ± 0.0007	0.1293 ± 0.0045
HR@10	**0.2143 ± 0.0027**	0.0701 ± 0.0022	0.2057 ± 0.0064
HR@20	**0.3202 ± 0.0046**	0.1175 ± 0.0012	0.3071 ± 0.0046
NDCG@5	**0.0871 ± 0.0020**	0.0258 ± 0.0004	0.0844 ± 0.0031
NDCG@10	**0.1126 ± 0.0011**	0.0352 ± 0.0009	0.1089 ± 0.0030
NDCG@20	**0.1392 ± 0.0018**	0.0471 ± 0.0007	0.1344 ± 0.0026
MRR	**0.0980 ± 0.0014**	0.0353 ± 0.0006	0.0953 ± 0.0023
Yelp (sparse, 104,072 users, 54,056 items)			
HR@5	0.0146 ± 0.0005	0.0172 ± 0.0002	**0.0190 ± 0.0005**
HR@10	0.0257 ± 0.0006	0.0299 ± 0.0004	**0.0331 ± 0.0007**
HR@20	0.0451 ± 0.0012	0.0517 ± 0.0005	**0.0562 ± 0.0009**
NDCG@5	0.0090 ± 0.0003	0.0106 ± 0.0001	**0.0119 ± 0.0003**
NDCG@10	0.0125 ± 0.0003	0.0147 ± 0.0001	**0.0164 ± 0.0004**
NDCG@20	0.0174 ± 0.0004	0.0202 ± 0.0001	**0.0222 ± 0.0004**
MRR	0.0136 ± 0.0003	0.0153 ± 0.0000	**0.0169 ± 0.0003**
Amazon-Books (very sparse, 156,213 users, 112,362 items)			
HR@5	0.0265 ± 0.0008	0.0146 ± 0.0002	**0.0433 ± 0.0004**
HR@10	0.0432 ± 0.0014	0.0242 ± 0.0003	**0.0651 ± 0.0007**
HR@20	0.0683 ± 0.0020	0.0401 ± 0.0003	**0.0947 ± 0.0010**
NDCG@5	0.0168 ± 0.0006	0.0095 ± 0.0001	**0.0285 ± 0.0004**
NDCG@10	0.0222 ± 0.0008	0.0126 ± 0.0001	**0.0355 ± 0.0004**
NDCG@20	0.0285 ± 0.0009	0.0166 ± 0.0001	**0.0429 ± 0.0005**
MRR	0.0209 ± 0.0007	0.0127 ± 0.0001	**0.0321 ± 0.0004**

5.1 Ablation study: which knobs matter, and where

Table 2 reports the ablation grid and Figure 3 visualises the same numbers as paired HR@10 / NDCG@10 bar charts; significance markers in the table are paired t-tests against the in-ablation reference variant (`sasrec_full`) at the same 50-epoch / 3-seed budget. Five patterns stand out.

Alignment heads matter asymmetrically. On Amazon-Books, removing the user-alignment head ($\lambda_u = 0$) sharply hurts performance (-32.0% NDCG@10, $p < 0.01$), while on Yelp it is neutral ($+2.1\%$, n.s.). Removing the item-alignment head ($\lambda_i = 0$) damages both datasets: -8.7% on Amazon-Books ($p < 0.05$) and -15.7% on Yelp ($p < 0.05$). This asymmetry reflects dataset structure: Yelp user profiles are short, formulaic metadata, so semantic signal resides in items; Amazon-Books user profiles derive purely from reading history, making user alignment essential.

λ sweep is bell-shaped. The default $\lambda = 0.1$ sits near the performance peak on all datasets. On Amazon-Books, the curve is sharp: $\lambda = 0.05$ is within -1.5%, but $\lambda = 1.0$ collapses by -19.0% ($p < 0.01$). On ML-1M, where alignment adds no real value, smaller weights ($\lambda \in \{0.01, 0.05\}$) outperform default by $+6\%$, while large λ $(0.5, 1.0)$ proves catastrophic (-28%, -39%, $p < 0.01$).

InfoNCE temperature is robust. The τ-sweep stays within -10% to $+6\%$ across all datasets, indicating the alignment is largely temperature-insensitive. On Amazon-Books, $\tau = 0.2$ is statistically indistinguishable from the default $\tau = 0.1$.

Projection-head architecture matters only on text-rich data. The linear projection head matches or slightly trails the MLP head on Yelp and ML-1M, but underperforms on Amazon-Books (-18.6%, $p < 0.01$). We interpret this as the MLP providing extra capacity to absorb rich semantic targets without disturbing the underlying CF embedding during ranking.

Encoder dimension matters on rich text. Swapping `BGE-large-en-v1.5` for the smaller `MiniLM-L6-v2` costs -20.1% NDCG@10 on Amazon-Books ($p < 0.01$), but only minor, non-significant drops on Yelp (-3.3%) and ML-1M (-6.9%). Text-rich Amazon-Books benefits from the larger encoder; Yelp and ML-1M either lack sufficient textual richness or already saturate `MiniLM-L6-v2`'s 384-dimensional capacity.

6 Conclusion

We have shown that the contrastive cross-view alignment introduced by RLMRec [13] for graph-CF backbones transfers cleanly to a self-attentive sequential backbone [10], *provided* the underlying dataset is in the regime RLMRec was originally designed for: sparse interactions, rich item-side text. On Amazon-Books, adding two thin MLP projection heads and an InfoNCE objective on top of

Table 2: Ablation grid for SemanticSASRec, 3 seeds × 50 epochs, NDCG@10 mean ± std. Significance vs. in-ablation reference `sasrec_full`: $*$ p$<$ 0.05, $**$ p$<$ 0.01, $***$ p$<$ 0.001 (paired two-sided t-test). Δ% is the relative change in NDCG@10.

Variant	MovieLens-1M	Yelp	Amazon-Books
Loss components			
SemanticSASRec (full, ref.)	0.1042 ± 0.0019	0.0162 ± 0.0002	0.0348 ± 0.0007
no user-align ($\lambda_u = 0$)	0.0925 ± 0.0022 (-11.2%)*	0.0165 ± 0.0004 ($+2.1\%$)	0.0237 ± 0.0002 (-32.0%)**
no item-align ($\lambda_i = 0$)	0.1088 ± 0.0038 ($+4.5\%$)	0.0137 ± 0.0005 (-15.7%)*	0.0318 ± 0.0005 (-8.7%)*
no align ($\sim$ pure SASRec)	0.1084 ± 0.0026 ($+4.1\%$)	0.0121 ± 0.0004 (-25.3%)**	0.0213 ± 0.0010 (-38.8%)**
Alignment weight λ sweep			
$\lambda = 0.01$	0.1105 ± 0.0017 ($+6.1\%$)	0.0148 ± 0.0001 (-8.9%)**	0.0280 ± 0.0005 (-19.5%)**
$\lambda = 0.05$	0.1108 ± 0.0015 ($+6.4\%$)*	0.0165 ± 0.0002 ($+1.7\%$)	0.0343 ± 0.0002 (-1.5%)
$\lambda = 0.5$	0.0746 ± 0.0011 (-28.4%)**	0.0136 ± 0.0002 (-16.0%)**	0.0310 ± 0.0001 (-11.0%)*
$\lambda = 1.0$	0.0639 ± 0.0013 (-38.7%)***	0.0120 ± 0.0001 (-26.0%)***	0.0282 ± 0.0003 (-19.0%)**
InfoNCE temperature τ sweep			
$\tau = 0.05$	0.1012 ± 0.0012 (-2.9%)	0.0151 ± 0.0007 (-7.0%)	0.0325 ± 0.0005 (-6.6%)**
$\tau = 0.2$	0.1077 ± 0.0022 ($+3.4\%$)	0.0167 ± 0.0004 ($+3.1\%$)	0.0346 ± 0.0007 (-0.6%)
$\tau = 0.5$	0.1107 ± 0.0032 ($+6.3\%$)	0.0156 ± 0.0003 (-3.8%)	0.0315 ± 0.0007 (-9.5%)*
Architecture / encoder			
linear proj-head	0.1015 ± 0.0035 (-2.6%)	0.0160 ± 0.0002 (-1.2%)	0.0283 ± 0.0003 (-18.6%)**
`MiniLM-L6-v2` encoder	0.0970 ± 0.0027 (-6.9%)	0.0157 ± 0.0002 (-3.3%)	0.0278 ± 0.0004 (-20.1%)**

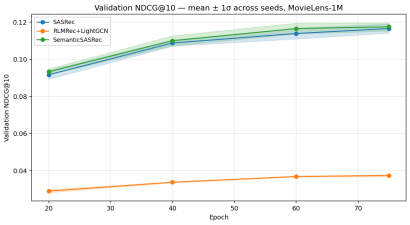
SASRec's last-position hidden state and item embedding table lifts NDCG@10 by +60.0% and HR@10 by +50.6% relative to a vanilla SASRec backbone with the same training configuration. On Yelp, the same recipe yields +30.8% NDCG@10 and +28.7% HR@10. On the dense, short-text MovieLens-1M dataset, the alignment loss is approximately neutral, with a small −3.3% NDCG@10 regression that we attribute to the very limited semantic content of ML-1M's profile text. our 14-variant ablation grid further shows that (i) the user- and item-alignment heads contribute *asymmetrically*, with the asymmetry tracking the relative richness of user- vs. item-side profile text on each dataset; (ii) the InfoNCE temperature is a relatively robust knob, with the τ-sweep staying within ±10% on every dataset; and (iii) the non-linear projection head and the choice of sentence encoder (`BGE-large-en-v1.5` over `MiniLM-L6-v2`) account for a non-trivial portion of the headline gain on text-rich datasets.

References

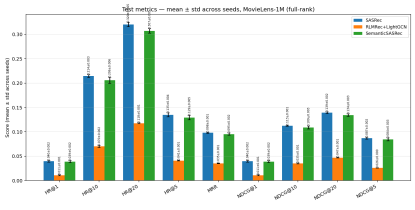
1. Bao, K., Zhang, J., Zhang, Y., Wang, W., Feng, F., He, X.: TALLRec: An effective and efficient tuning framework to align large language model with recommendation. In: Proceedings of the ACM Conference on Recommender Systems (RecSys). pp. 1007–1014 (2023)
2. Chen, T., Kornblith, S., Norouzi, M., Hinton, G.: A simple framework for contrastive learning of visual representations. In: Proceedings of the International Conference on Machine Learning (ICML). pp. 1597–1607 (2020)
3. Harper, F.M., Konstan, J.A.: The MovieLens datasets: History and context. ACM Transactions on Interactive Intelligent Systems **5**(4), 19:1–19:19 (2015). https://doi.org/10.1145/2827872

4. Harte, J., Zorgdrager, W., Louridas, P., Katsifodimos, A., Jannach, D., Fragkoulis, M.: Leveraging large language models for sequential recommendation. In: Proceedings of the ACM Conference on Recommender Systems (RecSys). pp. 1096–1102 (2023)
5. He, X., Liao, L., Zhang, H., Nie, L., Hu, X., Chua, T.S.: Neural collaborative filtering. In: Proceedings of the International Conference on World Wide Web (WWW). pp. 173–182 (2017). https://doi.org/10.1145/3038912.3052569
6. Hidasi, B., Karatzoglou, A., Baltrunas, L., Tikk, D.: Session-based recommendations with recurrent neural networks. In: International Conference on Learning Representations (ICLR) (2016)
7. Hou, Y., Li, J., He, Z., Yan, A., Chen, X., McAuley, J.: Bridging language and items for retrieval and recommendation. In: arXiv preprint arXiv:2403.03952 (2024)
8. Hou, Y., Liao, Z., Li, Y., Zhao, W.X., McAuley, J.: LLaRA: Large language-recommendation assistant. In: Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR) (2024)
9. Hou, Y., Zhang, J., Lin, Z., Lu, H., Xie, R., McAuley, J., Zhao, W.X.: Large language models are zero-shot rankers for recommender systems. In: Proceedings of the European Conference on Information Retrieval (ECIR) (2024)
10. Kang, W.C., McAuley, J.: Self-attentive sequential recommendation. In: Proceedings of the IEEE International Conference on Data Mining (ICDM). pp. 197–206. IEEE (2018). https://doi.org/10.1109/ICDM.2018.00035
11. Li, X., Chen, B., Hou, L., Tang, R.: CTRL: Connect tabular and language model for CTR prediction. In: arXiv preprint arXiv:2306.02841 (2023)
12. van den Oord, A., Li, Y., Vinyals, O.: Representation learning with contrastive predictive coding. arXiv preprint arXiv:1807.03748 (2018)
13. Ren, X., Wei, W., Xia, L., Su, L., Cheng, S., Wang, J., Yin, D., Huang, C.: Representation learning with large language models for recommendation. In: Proceedings of the ACM Web Conference (WWW) (2024). https://doi.org/10.1145/3589334.3645458
14. Rendle, S., Freudenthaler, C., Schmidt-Thieme, L.: Factorizing personalized Markov chains for next-basket recommendation. In: Proceedings of the International Conference on World Wide Web (WWW). pp. 811–820 (2010). https://doi.org/10.1145/1772690.1772773
15. Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., Jiang, P.: BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In: Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM). pp. 1441–1450 (2019). https://doi.org/10.1145/3357384.3357895
16. Tang, J., Wang, K.: Personalized top-n sequential recommendation via convolutional sequence embedding. In: Proceedings of the ACM International Conference on Web Search and Data Mining (WSDM). pp. 565–573 (2018). https://doi.org/10.1145/3159652.3159656
17. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS). pp. 5998–6008 (2017)
18. Wang, L., Lim, E.P.: Zero-shot next-item recommendation using large pretrained language models. In: arXiv preprint arXiv:2304.03153 (2023)
19. Xi, Y., Liu, W., Lin, J., Zhu, J., Chen, B., Tang, R., Zhang, W., Zhang, R., Yu, Y.: Towards open-world recommendation with knowledge augmentation from large language models. In: Proceedings of the ACM Conference on Recommender Systems (RecSys) (2024)

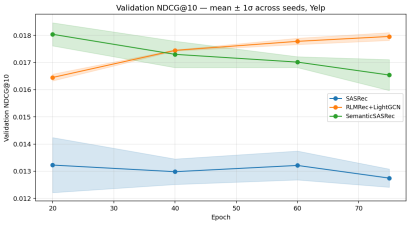
20. Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., Nie, J.Y.: BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. arXiv preprint arXiv:2402.03216 (2024)
21. Yelp Inc.: Yelp Open Dataset. https://www.yelp.com/dataset (2022), accessed 2025-03



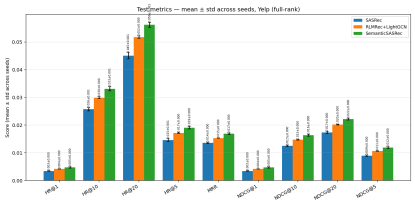
(a) ML-1M: validation NDCG@10 vs. epoch.



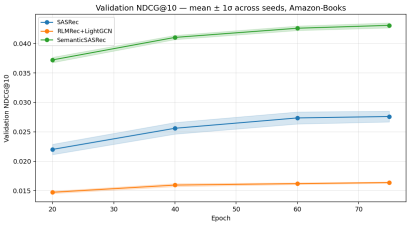
(b) ML-1M: test metrics.



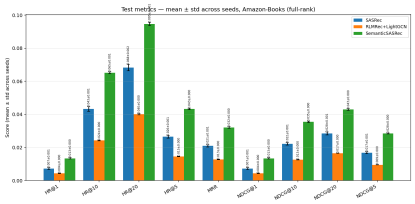
(c) Yelp: validation NDCG@10 vs. epoch.



(d) Yelp: test metrics.

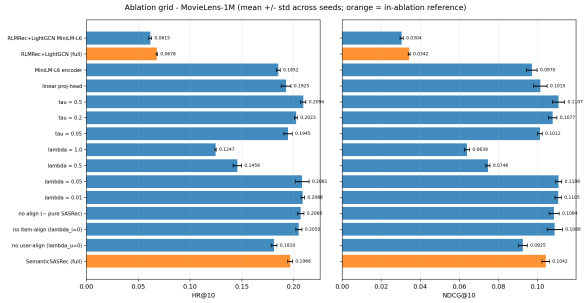


(e) Amazon-Books: validation NDCG@10 vs. epoch.

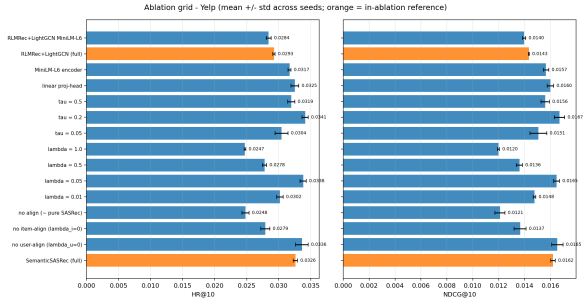


(f) Amazon-Books: test metrics.

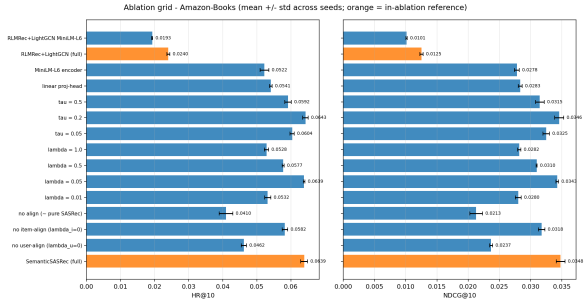
Fig. 2: Per-dataset comparison of the three models. **Left column:** validation NDCG@10 vs. training epoch (mean across five seeds, $\pm 1\sigma$ band). **Right column:** test-set metrics (mean $\pm$ standard deviation across five seeds, all nine metrics). The ordering agrees across both views and matches Table 1: SASRec is best on the dense, short-text ML-1M, while SemanticSASRec takes over on the sparser, text-richer Yelp and Amazon-Books; RLMRec+LightGCN trails on ML-1M because the bare LightGCN model has no notion of interaction order.



(a) MovieLens-1M. The reference `sasrec_full` variant (orange) sits in the middle of the SASRec variant cluster; turning the alignment heads off (`no_align`) or down ($\lambda \in \{0.01, 0.05\}$) even improves slightly, consistent with the dense dataset not benefiting from semantic alignment.



(b) Yelp. The reference `sasrec_full` (orange) dominates the cluster; removing the item-alignment head is the most expensive single ablation, and the $\lambda = 1.0$ collapse confirms that overweighting the alignment hurts.



(c) Amazon-Books. The reference `sasrec_full` (orange) is clearly the best variant. Both alignment heads matter — removing *either* drops NDCG@10 significantly — and the encoder / projection-head ablations are also costly.

Fig. 3: Ablation grid: HR@10 (left panel of each plot) and NDCG@10 (right panel of each plot), per dataset. Orange bars are the in-ablation reference variants (`sasrec_full` for SASRec variants; `lightgcn_full` for the LightGCN variants at the top). Wherever the reference bar dominates its variant cluster, the default alignment configuration is doing real work; wherever the reference is matched or beaten, the alignment is at best neutral.